

Gestion globale du programme

ZERDALI , Mohammed Karim – L3 ISI Groupe projet 5

Mots-clés

- Arduino
- Machine à états
- Sections
- Architecture

1. Objectifs et spécifications

Problématique

Concevoir un programme Arduino pilotant un robot autonome sur un parcours de plusieurs sections.

Objectif

Gérer l'enchaînement de tous les comportements (suivi ligne, tunnel, évitement, couleur, arrivée ...) de façon robuste et temps réel.

Contraintes

- Arduino UNO : mémoire et puissance de calcul limitées
- Bus I2C partagé entre 4 capteurs/actionneurs
- Robustesse aux faux positifs dans les transitions

2. Conception et modélisation

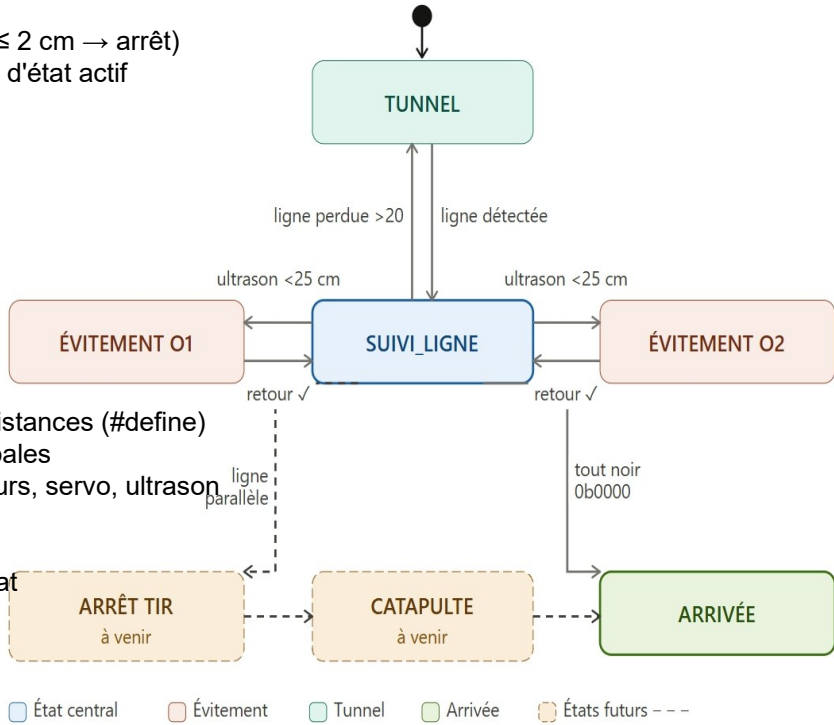
Boucle principale loop()

verifierUltrason() → sécurité prioritaire (dist ≤ 2 cm → arrêt)
switch(etatRobot) → dispatch vers la fonction d'état actif
SUIVI_LIGNE → gererSuiviLigne()
TUNNEL → gererTunnel()
EVITEMENT_O1 → evitementGauche()
EVITEMENT_O2 → evitementDroite()
ARRIVE → arreter()

Structure globale du programme

- 1 Librairies — Wire.h · Servo.h
- 2 Constantes — pins, adresses I2C, vitesses, distances (#define)
- 3 Machine à états — enum Etat + variables globales
- 4 Fonctions de base — pilotage moteurs, capteurs, servo, ultrason
- 5 setup() — initialisation unique au démarrage
- 6 loop() — boucle infinie, chef d'orchestre
- 7 Fonctions par état — un comportement par état

Machine à états finis (FSM) — 5 états codés + 2 à venir



3. Développements et mise en œuvre

Étapes suivies :

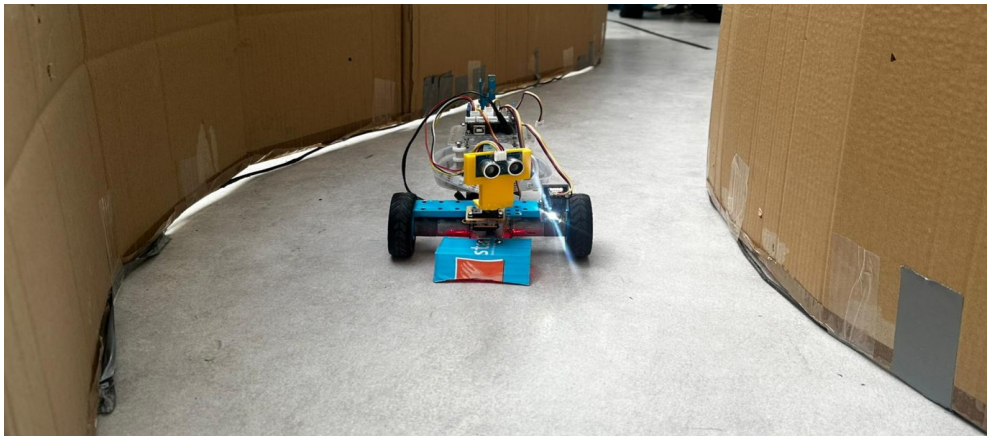
- ✓ Suivi de ligne avec correcteur proportionnel (KP)
- ✓ Traversée tunnel avec servo + correction paroi
- ✓ Contournement obstacle O1 (gauche) avec longage
- ✓ Contournement obstacle O2 (droite) avec longage
- ✓ Détection couleur TCS34725 + affichage LEDs
- ✓ Demi-tour + reprise ligne + arrêt final

Outils utilisés :

IDE Arduino, Moniteur série, Librairies : Wire.h, Servo.h, Adafruit_TCS34725, Adafruit_NeoPixel

Difficultés et solutions :

- Faux positifs tunnel dans virages → augmentation SEUIL_LIGNE_PERDUE
- Obstacle détecte trop tard → suppression condition servoPos, augmentation DIST_OBSTACLE
- LEDs couleurs inversées → passage de NEO_RGB à NEO_GRB
- Robot dévie en ligne droite → TRIM_G = -3
- Robot zigzag → KP = 0,0025 + zone morte 3000



Communication I2C :

Moteurs (0x66 / 0x68) · Capteur ligne (0x20)

Expérimentations et validation :

Chaque paramètre calibré par essais successifs sur piste réelle.
Validation état par état avant intégration dans la FSM.
Débogage via Serial.println() en temps réel.



4. Etat d'avancement et/ou Résultat

Sections réalisées et validées sur piste :

- ✓ Suivi ligne — Fonctionnel (correcteur P validé)
 - ✓ Tunnel — Fonctionnel (suivi de paroi droite confirmé)
 - ✓ Obstacle 1 (gauche) — Fonctionnel (contournement + longage)
 - ✓ Obstacle 2 (droite) — Fonctionnel (contournement + longage)
 - ✓ Détection couleur + LEDs — Fonctionnel
 - ✓ Demi-tour + reprise ligne — Fonctionnel
 - ✓ Arrêt sur zone noire — Fonctionnel
- 7 / 11 sections validées

Taches restantes :

- Section 9 — Arrêt précis + mesure distance poubelle
- Section 10 — Tir catapulte (servo lanceur)
- Section 11 — Circuit de chronométrage

Compétences acquises

- ✓ Programmation Arduino en C++
- ✓ Architecture logicielle (machine à états)
- ✓ Communication I2C (capteurs, moteurs)
- ✓ Asservissement proportionnel (PID)
- ✓ Intégration multi-capteurs (ultrason, couleur, servo)
- ✓ Débogage et calibration en temps réel



UNIVERSITÉ ÉVRY
PARIS-SACLAY

UFR Sciences
et Technologies